# chapter-6
## Synchronization Tools

**Race condition**- A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a race condition.

To guard against the race condition above, we need to ensure that only one process at a time can be manipulating the variable count. To make such a guarantee, we require that the processes be synchronized in some way.

**Critical Section**- Let us consider a system consisting of n processes $\{P_0, P_1, P_2, \ldots, P_{n-1}\}$. Each process has a segment of code, in which the process may be accessing and updating data that is shared with at least other process, is called a its critical section.

**Important features:**

- when one process is executing in its critical section, no other process is allowed to execution in its critical section.

Critical-section problem is to design a protocol that the processes can use to synchronize their activity so as to cooperatively share data.

Each process must request permission to enter its critical section.

- The section of code implementing the request is entry section; followed by exit section.

- The remaining code is remainder section

```
while (true){
    entry section
        critical section
    exit section
        remainder section
}
```

Fig: General structure of process

*** Critical section problem's solution must satisfy the following three requirements:

1. Mutual execution: If process Pi is executing in its criti section, then no other processes can be executing i their critical section.

2. Process – If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes not executing in their remainder sections can participate in decidin which will enter its critical section next and this selection can not be postponed indefinitely

3. Bounded waiting: There exists a bound, or limit, on the no. of times that other processes are allowed t enter their critical sections after a process has ma a request to enter its critical section and before that request is granted.

*** Two general approach for handling critical section problem:

1. Preemtive kernels — allows a process to be preempted while it is running in kernel mode.
2. Non-preemtive kernels — does not allow a process running in kernel mode to be preempted.

→ free from race condition, since one process is running.

*** Why favour a preemptive kernel over a non-preemptive one?

⇒ Preemptive kernel may be more responsive; since there is less risk that a kernel mode process will run for an arbitrarily long period before relinquishing the processor to waiting process.

Furthermore, a preemtive kernel is more suitable for real time programming as it will allow a real-time process to preempt a process currently running in the kernel

*** Peterson's Solution:

- is restricted to two processes that alternate execution betw their critical sections and remainder sections.
- The processes are $P_0$, $P_1$; $\{P_i, P_j; j = 1-j\}$
- requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

variable turn → indicates whose turn it is to enter the critical section.

if turn == i; then $P_i$ is allowed to enter variable flag → array is used to indicate if a process is ready to enter its critical section.

if flag[i] == true; $P_i$ is ready to enter.

Exp- Structure of processes $P_i$ in Peterson's solution.

```
while (true){
flag[i] = true;
turn = j;
while (flag[j] && turn == j)
;
/* critical section */
flag[i] = false;
/* remainder section */
}
```

Explanation: To enter the critical section, process Pi first sets flag[i] to be true and turn to the value j. Asserting, if Pj wants to enter the critical section, it can do so.
If both Pi and Pj wants to enter at the same time, turn will be set to both i and j at the same time. Only one of these assignments will last and other will occure that but will be overwritten immediately.
The eventual value of turn determines which process is allowed to enter its critical section first.
To prove that this solution is correct, we need to show that:
1. Mutual exclusion is preserved
2. The process requirement is satisfied.
3. The bounded waiting requirement is met.
To prove property 1, each process Pi enter the critical section only when flag[j] == false or turn == i. If both processes executes in the critical sections at the same time, then flag[0] == flag[1] == true. These two observations imply that P0 and P1 couldn't have successfully executed their while statements at the same time, since the value of turn can be either 0 or 1, but not both. So, one of the process will be successfully executed while other might have to execute at least one additional statement. So, mutual exclusion is preserved.
To prove properties 2 and 3, Pi can be prevented from entering the critical section only if flag[j] == true and turn == j. If Pj is not ready to enter the critical section,

flag[j] == false and turn == i; Pi can enter the critical sec
then. If Pj exits the critical section, it sets flag[j] == fal
and turn == i.
Thus since Pi doesn't change the value of the variable
turn, Pi will enter the critical section (progress) after a
most one entry by Pj (bounded waiting)

** Hardware support for synchronisation
Three hardware instructions that provide support for so
the critical-section problem:

1. Memory Barriers
memory model — How a computer architecture determi
what memory guarantees it will provide to an applic
program is known as its memory model.
Two categories:
1. Strongly ordered — where memory modification on one
processor is immediately visible to others.
2. Weakly ordered — may not be immediately visible to others

** Memory models vary by processor types, so assumptio
can't be made regarding visibility of modifications. To
address this issue memory barriers or memory fences
is used.

Memory Barrier/Fence: Computer architectures provide
instructions that can force any changes in memory to
propagated to all other processors, thereby ensuring that

memory modifications are visible to threads running on other processors. Such instructions are known as memory barriers.

## 2. Hardware Instructions

- Special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words atomically.
  - test_and_set() } executes atomically
  - compare_and_swap() }
- If two test_and_set() or two compare_and_swap() instructions are executed simultaneously each on different core, they will be executed sequentially in some arbitrary order.
- mutual exclusion implemented by declaring a boolean variable lock;
  - boolean variable lock; initialized to false → test_and_set()
  - global variable lock; initialized to 0 → compare_and_swap()

## 3. Atomic Variables

- provides atomic operations on basic data types such as integers and booleans.
- can be used to ensure mutual exclusion in situations where there may be a data race on a single variable while it's being updated, as when a counter is incremented.
- systems that supports atomic variables, provide special atomic data types as well as functions for accessing and manipulating atomic variables.

## Mutex Locks

- Higher-level software tool to be solve critical section prob
- mutex → Mutual Exeecution
- A process must acquire the lock before entering the critical section.
  acquire() function → acquires the lock
- A process releases the lock when it exits the critical section.
  release() → releases the lock.
- A mutex lock has a boolean variable available; whose value indicates if the lock is available or not.
- If the lock is available, a call to acquire() succeeds and the lock is then considered avail unavailable.
- A process that attempts to acquire an unavailable lock is blocked until the lock is released.

```
while (true) {
        [acquire lock]
            critical section
        [release lock]
            remainder section
}
```

Fig: Solution to critical section problem using mutex lock

Definition of acquire():

```
acquire() {
  while (!available)
    ; /* busy wait */
  available = false;
}
```

Definition of release():

```
release() {
  available = true;
}
```

*Lock contention

Contended - A lock is considered if a thread blocks while trying to acquire the lock.

Uncontended - If a lock is available when a thread attempts to acquire it, the lock is considered uncontended.

High contention - a relatively large numbers of threads attempting to acquire the lock.

Decreasing overall performance of concurrent applications.

Low contention - relatively small no: of threads attempting to acquire the lock.

Spinlocks - the locking mechanism of choice on multiprocessor systems when the lock is to be held for a short duration.

## Semaphores

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard operations: wait() and signal().

Definition of wait():

```
wait(S){
    while(S<=0)
      ; // busy wait
      S--;
    }
```

Definition of signal():

```
signal(S){
    S++;
    }
```

- All modifications to the integer value of S in the wait() and signal() operations must be executed atomically i.e. when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

## Semaphore usage

Binary semaphore: The value of binary semaphore can range only between 0 and 1. Thus, binary semaphore behaves similarly to mutex lock. On systems that do not provide mutex locks, binary ~~lock~~ semaphores can be used to provide mutual exclusion.

Counting semaphore: The value of a counting semaphore can range over an unrestricted domain. This can be used to control access to a given resource consisting of a finite no. of instances.

- We can also use semaphores to solve various synchronization problems. For example, two concurrently running process $P_1$ and $P_2$ with statement $S_1$ and $S_2$, where $S_2$ is to be executed only after $S_1$ has completed. Letting $P_1$ and $P_2$ share a common semaphore synch; In process $P_1$,

```
S1;
signal(synch);
```

In process $P_2$,

```
wait(synch);
S2;
```

## Semaphore Implementation

- To overcome busy waiting, we can modify the wait() and signal definition.
- When a process executes wait() operation and finds that the semaphore value is not positive, it must wait. However, rather than engaging in busy waiting, the process can suspend itself.
- The suspend operation places the process into a waiting queue associated with the semaphore and the state of the process is switched to the waiting state.

- A process that is suspended, waiting on a semaphores, should be restarted when some other process executes a signal() operation. The process is restarted by a wakeup() operation, which puts the process into ready state. Then it is placed in the ready queue.
- we define semaphore as:

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

Each semaphore has an integer value and a list of processes list.

- sleep() operation suspends the process and wakeup(P) resumes the execution of a suspended process P.

wait() definition:

```
wait(semaphore *S) {
    s->value--;
    if (s->value < 0) {
        add this process to s->list;
        sleep();
    }
}
```

signal definition:

```
signal(semaphore *s){
 s->value++;
 if(s->value<=0){
 remove a process P from s->list;
 wakeup(P);
 }
}
```

Monitors:

When the mutex lock and semaphores are used incorrectly to solve a critical section problem, some timing errors may occur

i) when a program interchanges the order in which the wait() and signal() operations on the semaphore mutex are executed.

```
signal(mutex);
----
critical section
----
wait(mutex);
```

ii) When a program replaces signal(mutex) with wait(mutex)

```
wait(mutex)
----
critical section
----
wait(mutex);
```

(iii) When wait() or signal() are both are omitted. Then the mutual execution is violated or the process will permanently block.

Strategy for dealing with such errores is to incorporate high-level synchronization construct - the monitore type.

## Monitore Usage

Abstract Data Type (ADT) - encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of ADT.

Monitore type - A monitore type is an ADT that includes a set of programmer defined operations that are provided with mutual exclusion within the mon - also declares the variables whose values define the state of an instance of that operate on the variab

Syntax of monitore

```
monitore monitorename {
    /* shareed variable declaration */
    function P1 (...) {
        ...
    }
    :
    function Pn (...) {
    }
    initialization-code (...) {
    }
}
```

## Implementing a monitore using Semaphorees

- For each monitore, a binary semaphore mutex (initialized to 1) is provided to ensure mutual exclusion.
- A process must execute wait(mutex) before entering the monitore and must execute signal(mutex) after leaving the monitore.
- An additional binary semaphore next, initialized to 0 is used to suspend the process.
- An integer variable next_count is also provided to count the number of processes suspended on next.

```
wait(mutex);
  ....
  body of F
  ....
if (next_count > 0)
  signal(next);
else
  signal(mutex);
```

Mutual exclusion within a monitore is ensured.

## Resuming Processes within a monitore

- which of the suspended processes should be resumed next?
  - may use first_come, first_served (FCFS) ordering
    - no adequate for all
- conditional wait construct can be used, which has the form
  - x.wait(c);

where c is an integer expression that is evaluated when the wait() operation is executed.

c = priority no.

process with smallest priority no. is resumed next.

### Liveness

- refers to a set of properties that a system must satisfy to ensure that processes make progress during their execution life cycle.

**Liveness failure** - A process waiting indefinitely for another process to complete its execution is called liveness failure.

- Example: Deadlock, Priority inversion.

**Deadlock** - The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for execution of a process, when such a state is reached, these processes are said to be deadlocked.

**Priority inversion**

- when higher priority process needs to read or modify kernel data, but a lower priority process is accessing the kernel resource, the higher priority process then has to wait as kernel is protected with a lock. But if the lower priority

process is preemptive in favor of another higher priority process, it becomes very complicated. This liveness problem is known as priority inversion.

- it occurs only in systems with more than two priorities.
- can be avoided by implementing a priority inheritance protocol.

# Chapter-7
## Synchronization Examples

- classic Problems of synchronization

1. Bounded buffers Problem
2. The readers-writers Problem
3. The dining-Philosophers Problem

- The dining-Philosophers Problem

Problem statement:

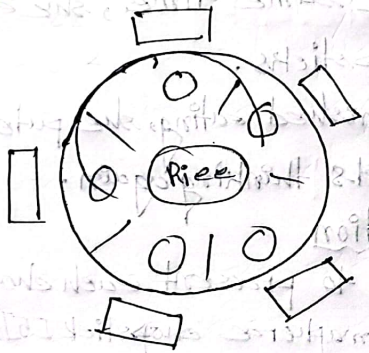



Fig: The situation of the dining philosophers

- consider five philosophers who spend their life thinking and eating. They share a circular table surrounded by five chairs, each belonging one to one philosopher.
- In center, there is a rice bowl and the table is laid with five single chopsticks.
- when a philosophers thinks, she doesn't interact with her colleques.

- From time to time, a philosoper gets hungry and tries to pickup the two chopsticks that are closests to her.
- A philosoper may pick one chopstick at a time. She cant pick up a chopstick that is already in the hand of a neighbor.
- when a hungry philosoper has both her chopsticks at the same time, she eats without releasing the chopsticks.
- when she is finished eating, she putes down both chopsticks and starets thinking again.

## Semaphore solution

- one situation is to present each chopstick with a semaphore. `semaphore chopstick[5];`
- A philosoper tries to grap a chopstick by executing a wait() operation on that semaphore.
- She releases her chopsticks by executing the signal() operation on the appropriate semaphores.
- Although this guarantees no two neighbores are eating simultaneously, it can create deadlock. When all five philosopers become hungry at the same time and each grabs her left chopstick; when they tries to grab their right chopsticks still be delayed forever.

— To solve deadlock problem:
i) Allow at most four philosophers to be setting simultaneously.
ii) Allow a philosoperes to pick up here chopstick only if both chopstick is available.
iii) use an assymmetric solution — an odd-numbered philosopherc picks up here left chopstick first and then right; whereas even numbered philosopere picks up here right chopstick first and then left.
— A deadlock free solution doesn't necessarily eliminate the possibility of starvation. One of the philosoperes would starve to death.

Monitore solution
— This ~~impress~~ impases a restriction that a philosoperc may pick up here chopstick only both of them are available.
— To code this, we may find three states in which the philosoperes might be —
Thinking, Hungry, Eating
— we also need condition self[5] which allows a philosoperc to delay herself when she is hungry but is unable to obtain the chopsticks she needs.

- Each philosopere must invoke the operation pickup() before starting to eat. This may result in the suspension of the process. After the successful completion of the operations the philosopere may eat.
- following this, the philosopere invokes putdown() operation

Dining_philosoperes.pickup(i);

eat

Dining_philosoperes.putdown(i);

# chapter-8
## Deadlocks

Deadlock- A thread request resources; if the resources are not available at the time, the thread enters a waiting state. Sometimes, a waiting thread can never again change state, because the resources it has requested are held by others by other waiting threads. This situation is called a deadlock.

System model

System- A system consists of a finite number of resources to be distributed among a number of competing threads.

Under the normal mode of operation, a thread may utilize a resource in only the following sequence:

1. Request- The threads request the resource. If the request can not be granted immediately, then the requesting thread must wait until it can acquire the resource.
2. Use: The thread can operate on the resource.
3. Release: The thread releases the resource after using it.

## Deadlock in Multithreaded Applications

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;
```

Here two mutex locks are created.
Next two threads are created. If thread_one attempts to acquire the mutex locks in order (1) first_mutex (2) second_mutex. At the same time, thread_two attempts to acquire the mutex locks in order (1) second_mutex (2) first_mutex. Deadlock is possible if thread_one acquires first_mutex while thread_two acquires second_mutex.

## Livelock

- another form of liveness failures; similar to deadlock.
- occurs when a thread continuasly attempts an action that fails.
- can generally be avoided by having each thread retry the failing operation at random times.

## Deadlock characterization

Necessary conditions:

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. Mutual exclusion — At least on resource must be held in a nonsharable mode.

2. Hold and wait – A thread must be holding at least one resource and waiting to acquire additional resources that are currently being held by other threads.

3. No preemption – Resources can not be preempted; i.e a resource can be released only voluntarily by the thread holding it, after that thread has completed its task.

4. Circular wait – A set $\{T_0, T_1, \ldots T_n\}$ of waiting threads must exist such that $T_0$ is waiting for a resource held by $T_1$, $T_1$ for $T_2$, ..., $T_{n-1}$ for $T_n$ and $T_n$ is waiting for $T_0$.

Resource Allocation Graph:

- A graph that describes the deadlock situation.
- consists of a set of vertices V and set of Edges E.
- set of vertices is partitioned into two different types of nodes.

$T = \{T_1, T_2, \ldots T_n\}$ → active threads

$R = \{R_1, R_2, \ldots T_m\}$ → resource type

Thread $T_i$ → Resource $R_j$ = Request edge

Resource $R_j$ → Thread $T_i$ = Allocation Edge

- Thread $T_i$ is represented by a circle and resource type $R_j$ is represented by a rectangle.

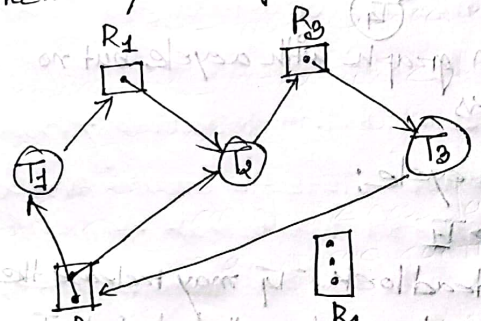
Thread states:

(i) $T_1$ is holding an instance of $R_2$ and is waiting fore an instance of $R_1$.

(ii) $T_2$ is holding an instance of $R_1$ and $R_2$; is waiting fore an instance of $R_3$

(iii) $T_3$ holding an instance of $R_3$

- There each resource has exactly one instance type then a cycle implies that a deadlock occured.




17-18 6(b)

Fig: Resource allocation graph with a deadlock.

- If $T_3$ requests an instance of $R_2$, since no instance is available, we added a request edge $T_3 \rightarrow R_2$.
- At this point, two minimal cycles exists in the system:

$T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$

$T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2$

Here, threads $T_1$, $T_2$ and $T_3$ are deadlocked.

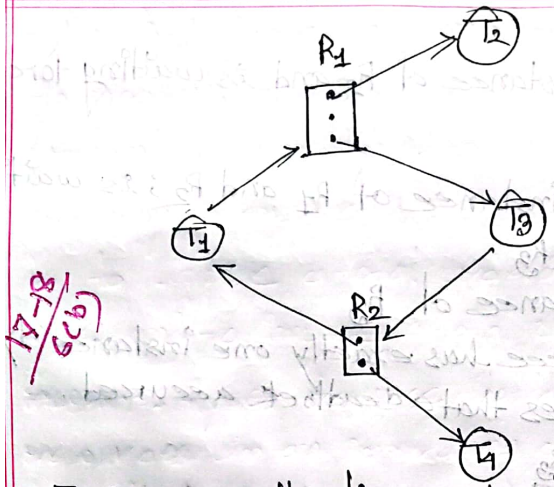



Fig: Resource allocation graph with a cycle but no deadlocks

- Here, there's also a cycle:

  $T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$

- However, there's no deadlock. $T_4$ may release the instance of $R_2$ and that can be allocated to $T_3$, breaking the cycle.

## Methods for Handling Deadlocks:

We can deal with the deadlock problem in one of three ways:

i) We can ignore the problem altogether and pretend that deadlocks never occur in the system.

(ii) we can allow the system use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.

(iii) We can allow the system to enter a deadlocked state, detect it and recover.

Deadlock Prevention

- By ensuring that at least one of the four conditions cannot hold, we can prevent the occurrance of a deadlock.
- The mutual exclusion condition must hold. We can not prevent deadlocks by denying the mutual exclusion condition because, some resources are intrinsically nonsharable.
- To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a thread requests a resource, it does not hold any other resources.
- To ensure that the two preemption condition doesn't hold, we can use the following protocol:
If the thread is holding some resources and requests another resource that can not be immediately allocated to it, then all resources the thread is currently holding are preempted.

— The first three conditions are very impractical to prevent. The circular wait condition can be prevented by solutions.

One way to ensure that this condition never holds is to impose a total ordering of all resources types and to ~~impose a total ordering of all resources types~~ require that each thread requests resources in an increasing order of enumeration.

## Deadlock Avoidance

— A deadlock avoidance algorithm dynamically examing the resource-allocation state to ensure that a circular wait condition can never exit.

— Resource allocation state is defined by the number of available and allocated resources, and the maximum demands of the threads.

## Safe state:

— A state is safe if the system can allocate resources to each thread in some order and still avoid deadlocks.

— A system is in a safe state if there exists a safe sequence.

- A sequence of threads $\langle T_1, T_2, \ldots, T_n \rangle$ is a safe sequence for the current allocation state if, for each $T_i$, the resource requests that $T_i$ can still make, can be satisfied by the currently available resources plus the resources held by all $T_j$ with $j < i$.
- A deadlocked state is an unsafe state.
- Not all unsafe states are deadlocks; however, An unsafe state may lead to a deadlock.

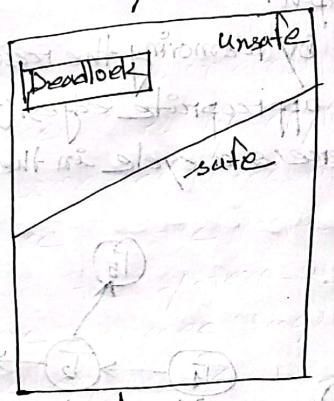



Fig: Safe, unsafe and deadlocked state space

- Two algorithms
  i) Resource-allocation-graph Algorithm (single instance)
  ii) Banker's Algorithm (multiple instance)

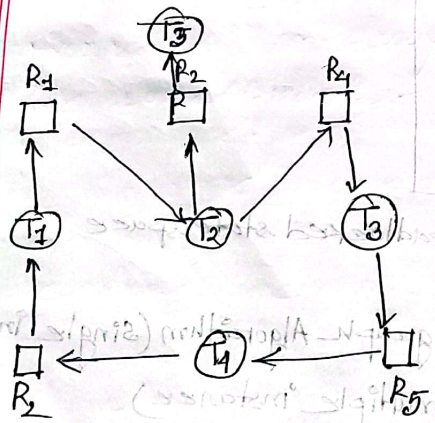
## Deadlock Detection

The system may provide:

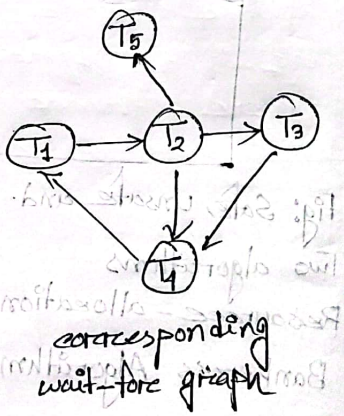
i) An algorithm that examines the state of the system to determine has occured.

ii) An algorithm to recover from the deadlock.

- Single instance of each Resource Type
- Uses variant of the resource allocation graph called a wait for graph.
- we obtain the graph by removing the resource nodes and collasping to appropriate edges. ($T_i \rightarrow T_j$)
- Deadlock exists if there is a cycle in the graph.




Resource allocation graph




corresponding wait-for graph

Several Instances of a Resource Type

employs several data structures:

i) Available — no. of available resources

ii) Allocation — no. of resources of each type currently allocated to each thread.

iii) Request — current request of each thread.

Detection Algorithm Usage

1. How often is a deadlock likely to occur?

⇒ If the deadlock occurs frequently, then the detection algorithm should be invoked frequently.

2. How many threads will be affected by deadlock when it happens?

⇒ Resources allocated to deadlocked threads will be idle until the deadlock can be broken. The no. of threads involved in the deadlock may grow.

Recovery from Deadlock

Process and thread termination

i) Abort all deadlocked processes — This method will clearly break the deadlock but at great expenses.

ii) Abort one process at a time until the deadlock cycle is eliminated.

## Resource Preemption

- To eliminate deadlocks, we successively preempt some resources from processes and give these to other processes until the deadlock cycle is broken.
- Three issues needs to be addressed:

i) Selecting a victim – which process & resource are to be preempted.

ii) Rollback – what should be done with the preempted process

iii) Starvation – How can we guarantee that resources will not always be preempted from the same process.

2017-18

⑤ @ Why synchronization tools are needed to ensure concurrent access? How to solve critical section problem using Peterson's solution? — (7)

Ans: Synchronization tools are necessary to ensure safe and coordinated concurrent access to shared resources (like memory, files or variables) in multi-threaded or multi-process environments. Without synchronization, multiple threads or processes can attempt to read from or write to shared resources simultaneously, leading issues like:

1. Race condition: When two or more threads/processes try to change the same data at the same time, the result may depend on the unpredictable timing of the processes. This leads to inconsistance or incorrect results.
2. Deadlocks: Without proper synchronization, processes may hold on to resources while waiting for others, leading to a situation where they are stuck, each waiting for the other to release the resources, preventing all progress.
3. Data inconsistency: Concurrent access to shared data without synchronization can lead to data corruption, where one process writes over the changes made by another, resulting in inaccurate or incomplete data.

4. Thread safety: Synchronization ensures that shared data is accessed in a safe manner, preventing threads from interfering with one another's operations.

To address these problems, synchronization tools such as locks (mutexes), semaphores, condition variables, monitors and barriers are used. They coordinate access, ensuring that only one thread or process can access critical sections of code or shared resources at any given time, thereby maintaining data integrity and ensuring predictable outcomes.

Peterson's solution is a classic software-based method for achieving mutual exclusion in concurrent programming. It allows two processes to share a single-use resource without conflict, using only shared memory for communication. It works by having two shared variables: one indicating "interest" to enter the critical section and another indicating which process should be given priority if both are interested.

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. Let's assume

we have two processes, $P_0$ and $P_1$. Here's how Peterson's solution can be implemented for these processes:

Shared variables:

- flag: An array of boolean where flag[i] indicates if process $P_i$ is interested in entering the critical section.
- turn: An integer variable that indicates whose turn it is to enter the critical section.

Process $P_0$:

```
flag[0] = True
turn = 1
while flag[1] and turn == 1:
flag[0] = False
```

Process $P_1$:

```
flag[1] = 0
turn = 1
while flag[0] and turn == 1:
flag[1] = fals
```

Explanation

- Entry section: A process sets it's flag to True to show its interest in entering the critical section and gives the turn to the other process. It then waits (busy wait) until the other processes is either not interested (flag[other] == False) or its own turn comes up.

• critical section: The critical code or the shared resource is accessed here.

• Exit section: The process sets it's flag to False to indicate that it has finished using the shared resource and is no longere interested in the critical section.

• Remainder Section: The non-critical portion of the process's code.

We now prove that the solution is correcet. We need to show that:

1. Mutual exclusion is preserved
2. The progress requirement is satisfied
3. The ~~bound~~ bounded-waiting requirement is met.

• Mutual exclusion: Is satisfied as at any point, the "turn" variable and the "flag" array prevent both processes from entering the critical section simultaneously, the one that did not set the "turn" last will wait.

• Progress: When a process leaves the critical section it does not interfere with the other process entering the critical section. Only processes that are trying to enter the critical section are involved in the decision, and the decision is made in a finite number of steps.

• Bounded-waiting: The alternation of turns ensures that the wait is bounded. After a process expresses it's interest and gives the turn to the other process has finished it's critical section or if the other process is not interested.

Limitations:

• Peterson's solution works for only two processes.
• It assumes the processes can access shared variables atomically (without interference from other processes).

This solution, while simple and elegant for two processes, is less practical for systems with more processes, but it serves as a foundational example of synchronization in concurrent programming.

(6) What are mutex locks? State the solution of critical section problem using mutex locks? — (6)

Mutex lock: A mutex (mutual exclusion) lock is a synchronization mechanism used in concurrent programming to ensure that only one thread or process can access a shared resource at a time. This prevents race conditions, which occur when multiple threads attempt to read or write shared data simultaneously.

Solution to the critical section problem using mutex locks:

The critical section problem ensuring that multiple processes or threads can operate on shared resources without causing inconsistencies. A solution using mutex locks can be described in following steps:

1. Initialization: A mutex is created to control access to the critical condition.
2. Entering the critical section:
- A process must acquire the mutex lock before entering the critical section. This is done using locking function.
- If the mutex is already locked using another process, the requesting will be locked blocked until the mutex is released.
3. Executing the critical section: Once the mutex is acquired, the process can safely execute the code within the critical section, where it can access and modify shared resources.
4. Exeting the critical section: After the critical section is executed, the process releases the mutex lock, allowing other processes to acquire it.

Pseudocode Example:

```
mutex lock;
procedure process() {
lock(mutex lock);
unlock
```

```
while (true) {
acquire lock
  critical section
release lock
remainder section
}
```

Fig: Solution to critical section using mutex locks

We use the mutex lock to protect critical sections and thus prevent race conditions. That is, a process must acquire the lock before entering critical section; it releases the lock when it exits the critical section. The acquire() function acquires the lock and the release() function releases the lock.

A mutex lock has a boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to acquire() succeeds, and the lock then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released.

Using mutex locks effectively helps in managing concurrency and ensuring the integrity of shared resources in a multithreaded environment.

@Define liveness in process execution life cycle.

Answer: Liveness refers to a set of properties that processes make progress during their execution life cycle. A process waiting indefinitely under the ~~same~~ circumstances just described in an example of a "liveless failure".

There are many different ~~types~~ forms of liveness failure; however, all are generally characterized by poore performance and responsiveness. A very simple example of a liveness failure, ~~especially if a process may loop on~~ is an infinite loop.

6. @ Write a formal definition of deadlock in a multithread application. —(2)

⇒ Deadlock in a multithread application is a situation where two or more threads are unable to proceed with their execution because each thread is waiting for a resource held by another thread in the set. This condition occurs when the following four necessary conditions hold simultaneously: mutual exclusion, hold and wait, no preemption, and circular wait. In essence, deadlock results in a complete halt in progress as the involved threads remain indefinitely blocked.

(b) Illustrate and explain the resource-allocation graph, resource-allocation graph with deadlock, and resource-allocation graph with a cycle but no deadlocks. —(6)

⇒ The graph that describes the deadlock situation is called resource-allocation graph.

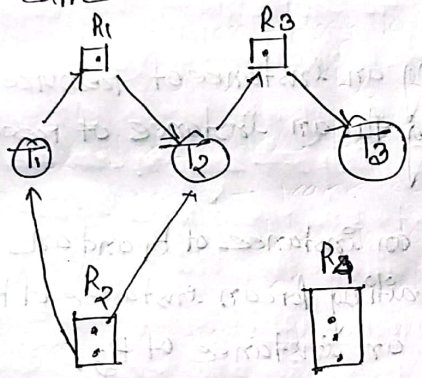



Figure1: Resource allocation graph.

The resource-allocation graph shown in the figure-1 depicts the following situation.

The sets T, R, E:

$T = \{T_1, T_2, T_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{T_1 \to R_1, R_1 \to T_2, T_2 \to R_3, R_3 \to T_3, R_2 \to T_1\}$

Resource instances:

- one instances of resource type $R_1$
- Two instances of resource type $R_2$
- one instances of resource type $R_3$
- three instance of resource type $R_4$

Thread status:

- Thread $T_1$ is holding an instance of resource type $R_2$ and is waiting for an instance of resource type $R_1$
- Thread $T_2$ is holding an instance of $R_1$ and an instance of $R_2$ and is waiting for an instance of $R_3$.
- Thread $T_3$ is holding an instance of $R_3$.

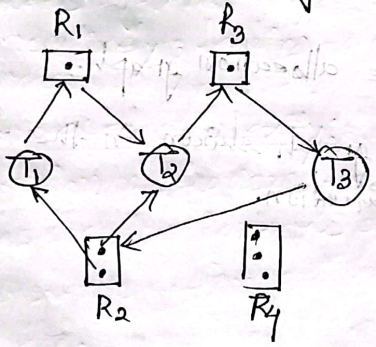



Figure-2: Resource allocation graph with a deadlock.

In the figure-2, two minimal cycles exist in the system,

$T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$

$T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2$.

Threads $T_1, T_2, T_3$ are deadlocked. Thread $T_2$ is waiting for the resource $R_3$, which held by thread $T_3$. Thread $T_3$ is waiting for either thread $T_1$ or thread $T_2$ to release $R_2$. In addition, thread $T_1$ is waiting for thread $T_2$ to release resource $R_1$.

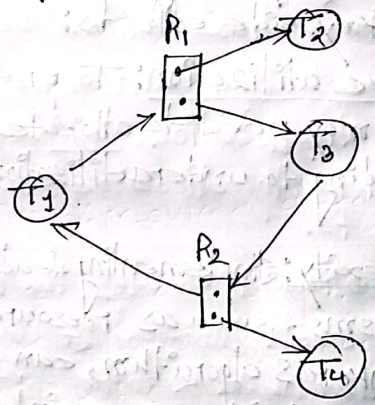



Figure-3: Resource-allocation graph with a cycle but no deadlock.

In figure-3 we have a cycle

$T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$

However there is no deadlock. Observe that

thread $T_4$ may release the instance of resource type $T_2$. That resource can then be allocated to $T_3$, breaking the cycle.

② What are the side effects of preventing deadlock? Describe the data structure which must be maintained to implement the banker's algorithm for deadlock avoidance — ⑥

⇒ Preventing deadlocks in a system can have several side effects, which can impact system performance, complexity and resource utilization. Here are some key side effects:

1. Reduce resource utilization: To prevent deadlocks, systems may need to allocate resources conservatively, leading to underutilization of available resources.
2. Increased Complexity: Implementing deadlock prevention mechanisms, such as resource allocation graphs or the Banker's algorithm, can complicate the system's design and increase maintenance overhead.
3. Potential for starvation: While preventing deadlocks, some threads may be indefinitely delayed if resource allocation policies favor certain threads over others, leading starvation.

4. Performance overhead: Additional checks and algorithms to monitore or manage resource allocation can induce overhead, potentially degrading overall system performance.

5. Less flexibility: Stick policies to avoid deadlocks can reduce the systems ability to adapt to varying workloads, making it less efficient in dynamic environment.

6. Complecated recovery: In systems where deadlock prevention is enforced, recovering from temporary states of contention can be more complex and may require intricate logic.

While preventing deadlocks enhances system stability, it can induce challanges related to resource utilization, complexity, performance and potential starvation.

~~The resour~~ The bankers's Algorithm is used for deadlock avoidance to a resource-allocation system with multiple instances of each resource type. Several data structures must be maintained to implement the bankers's algorithm. These data structures encode the state of the resource-allocation system. We need the following data structures, wherein n is the number of threads

in the system and m is the number of resources types:

• Available: A vector of length m indicates the number of available resources of each type. If Available[j] equals k, then k instances of resource type Rj are available.

• Max: An nxm matrix defines the maximum demand of each thread. If Max[i][j] equals k, then thread Ti may request at most k instances of resource type Rj.

• Allocation: An nxm matrix defines the number of resources of each type currently allocated to each thread. If Allocation[i][j] equals k, then thread Ti currently allocated k instances of resource type Rj.

• Need: An nxm matrix indicates the remaining resource need of each thread. If Need[i][j] equals k, then Ti may need k more instances of resource type Rj to complete its task. Note that Need[i][j] equals Max[i][j] − Allocation[i][j]

These data structures vary over time in both size and value.